SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Klasifikácia terénu zo signálov akcelerometra a gyroskopu

HNS – Zadanie 1

2025/2026 Bc. Dominik Kupec

Obsah

Úvod **3**

1 Dáta a ich spracovanie **4**
1.1 Databáza meraní 4
1.2 Rozdelenie na trénovaciu, validačnú a testovaciu množinu 4
1.3 Segmentácia na okná 4
1.4 Extrakcia príznakov pre SVM a MLP 5

2 Modely s ručne navrhnutými príznakmi **6**
2.1 SVM 6
2.2 MLP 6

3 Modely s automatickou extrakciou príznakov **10**
3.1 CNN 10
3.2 LSTM 10
3.2.1 Prvá verzia a jej zlyhanie 10
3.2.2 Úpravy modelu 10
3.2.3 Ladenie parametrov a štruktúry 13

4 Výsledky a porovnanie **19**
4.1 Súhrnné porovnanie modelov 19
4.2 Diskusia 20
4.3 Kontingenčné matice 20
4.4 Prínos prepracovania LSTM 23

5 Poznámky k metodike a obmedzenia **26**

6 Implementácia **27**

Záver **28**

Úvod

Cieľom zadania je klasifikovať typ terénu, po ktorom sa pohybuje robotická platforma, výlučne zo signálov inerciálnych snímačov – trojosového akcelerometra (ACC) a trojosového gyroskopu (GYR). Úloha zodpovedá reálnemu scenáru, v ktorom robot nemá k dispozícii kameru ani mapu a musí povrch rozpoznať „hmatom“, teda z vibrácií prenášaných do podvozku.

Zadanie požaduje porovnať dva odlišné prístupy ku klasifikácii:

- **modely s ručne navrhnutými príznakmi** – zo surového signálu sa najprv vypočítajú štatistické príznaky v časovej oblasti a tie vstupujú do klasifikátora (SVM, MLP),
- **modely s automatickou extrakciou príznakov** – sieť dostane surový priebeh signálu a reprezentáciu si vytvorí sama (LSTM, CNN).

Pre každý model bolo podľa zadania potrebné porovnať priebehy trénovania pri rôznych parametroch trénovania, porovnať viacero štruktúr siete vrátane vplyvu Dropout vrstvy, a najlepšiu konfiguráciu spustiť päťkrát na nezávislých rozdeleniach dát so štatistickým vyhodnotením úspešnosti, presnosti a senzitivity.

Dokument popisuje spracovanie dát, návrh a ladenie všetkých štyroch modelov, dosiahnuté výsledky a ich porovnanie. Osobitná pozornosť je venovaná modelu LSTM, ktorý v prvej verzii výrazne zaostával za ostatnými a bolo ho potrebné prepracovať; rozbor príčin a riešenia je v kapitole 3.2.

1 Dáta a ich spracovanie

1.1 Databáza meraní

Použitá bola databáza `outdoor_terrain_classification_database`. Obsahuje merania zo šiestich typov povrchu – betón (*concrete*), tráva (*grass*), kamienky (*pebbles*), piesok (*sand*), dlažobný kameň (*paving_stone*) a syntetická bežecká dráha (*running_track*).

Každý súbor je pomenovaný v tvare `SEN_x_y_z.csv`, kde `SEN` je typ snímača (ACC, GYR, MAG), `x` kód terénu (1–6), `y` kód rýchlosti (2 = 86 % PWM, 3 = 100 % PWM) a `z` číslo merania (session, 1–7). Na každý terén teda pripadá 14 meraní (7 session × 2 rýchlosti), spolu 84 meraní.

Do klasifikácie vstupujú snímače ACC a GYR, ktoré majú zhodnú vzorkovaciu frekvenciu 400 Hz, takže ich vzorky sú časovo zarovnané a dajú sa priamo spojiť do šiestich kanálov (ACC_X/Y/Z, GYR_X/Y/Z). Magnetometer (MAG) so 100 Hz by si vyžadoval prevzorkovanie a pre rozpoznávanie vibrácií povrchu nenesie podstatnú informáciu, preto použitý nebol.

1.2 Rozdelenie na trénovaciu, validačnú a testovaciu množinu

Rozdelenie prebieha na úrovni **celých meraní**, nie jednotlivých okien. Je to zásadné: okná z toho istého 4,5-sekundového merania sa prekrývajú a sú silno korelované. Keby sa rozdeľovalo až po narezaní na okná, okná z jedného merania by skončili súčasne v trénovacej aj testovacej množine, model by si fakticky zapamätal konkrétne meranie a úspešnosť by bola umelo nadhodnotená.

Z 14 meraní každého terénu sa náhodne vyberie 10 na trénovanie, 2 na validáciu a 2 na testovanie. Aby výsledky nezáviseli od jedného náhodného rozdelenia, vytvorených bolo **5 nezávislých rozdelení** (`split_00` až `split_04`) s pevne daným náhodným semienkom, takže sú reprodukovateľné. Skript navyše overuje, že sa žiadne meranie nevyskytuje vo dvoch podmnožinách naraz.

1.3 Segmentácia na okná

Každé meranie trvá 4,5 s (1800 vzoriek). Podľa zadania sa reže na okná dĺžky 1 s (400 vzoriek) s posunom 0,5 s (200 vzoriek), čo dáva $(1800 - 400)/200 + 1 = 8$ okien na meranie. Výsledkom je pole tvaru $(n_{\text{okien}}, 400, 6)$.

Tabuľka 1: Veľkosť jednotlivých množín pre jedno rozdelenie

Množina	Meraní	Okien	Okien na triedu
Trénovacia	60	480	80
Validačná	12	96	16
Testovacia	12	96	16

Trieda je teda dokonale vyvážená (rovnaký počet okien pre každý terén), čo zjednodušuje interpretáciu metrík – náhodné hádanie by dosiahlo 16,7 %. Zároveň je dataset veľmi malý: 480 trénovacích okien pochádzajúcich len zo 60 meraní. Táto skutočnosť ovplyvnila návrh všetkých modelov – prioritou bola regularizácia, nie kapacita siete.

1.4 Extrakcia príznakov pre SVM a MLP

Pre modely s ručnou extrakciou sa z každého okna počíta 9 príznakov v časovej oblasti na každý kanál:

- **MAV** – priemerná absolútna hodnota, $\frac{1}{N}\sum |x_i|$,
- **RMS** – efektívna hodnota, $\sqrt{\frac{1}{N}\sum x_i^2}$,
- **WL** – dĺžka priebehu, $\sum |x_{i+1} - x_i|$,
- **NZC** – počet prechodov nulou (s prahom proti šumu),
- **NSSC** – počet zmien sklonu (lokálnych extrémov),
- **WAMP** – počet zmien amplitúdy nad prahom,
- **MAX**, **MIN**, **PTP** – maximum, minimum a rozkmit.

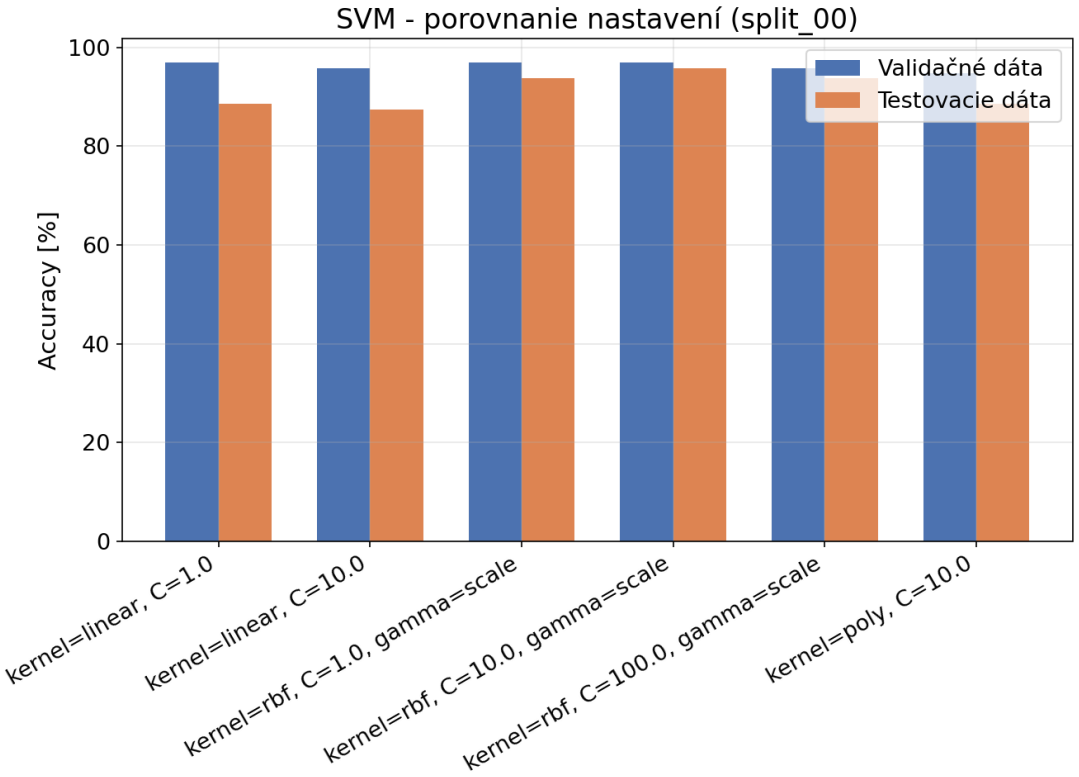
Pri 6 kanáloch vzniká vektor s $9 \times 6 = 54$ príznakmi. MAV, RMS a WL popisujú energiu vibrácií (mäkký piesok tlmí, dlažba budí ostré nárazy), NZC, NSSC a WAMP nepriamo zachytávajú frekvenčný obsah signálu.

Oba modely dostávajú *identický* vstup, takže rozdiel v ich výsledkoch pochádza výlučne z modelu a nie z odlišných dát. Príznaky sa škálujú pomocou `StandardScaler`, pričom parametre škálovania sa počítajú **len z trénovacej množiny** – inak by do modelu unikla informácia o testovacích dátach.

2 Modely s ručne navrhnutými príznakmi

2.1 SVM

Ako referenčný klasický model bol použitý `SVC` zo scikit-learn. Porovnaných bolo šesť nastavení – lineárne jadro s $C \in \{1, 10\}$, RBF jadro s $C \in \{1, 10, 100\}$ a polynomiálne jadro. Najlepšiu validačnú úspešnosť dosiahlo lineárne jadro s $C = 1{,}0$, čo naznačuje, že 54 príznakov oddeľuje triedy takmer lineárne a zložitejšie jadro prináša len riziko preučenia.



Obr. 1: SVM – porovnanie nastavení jadra a parametra C

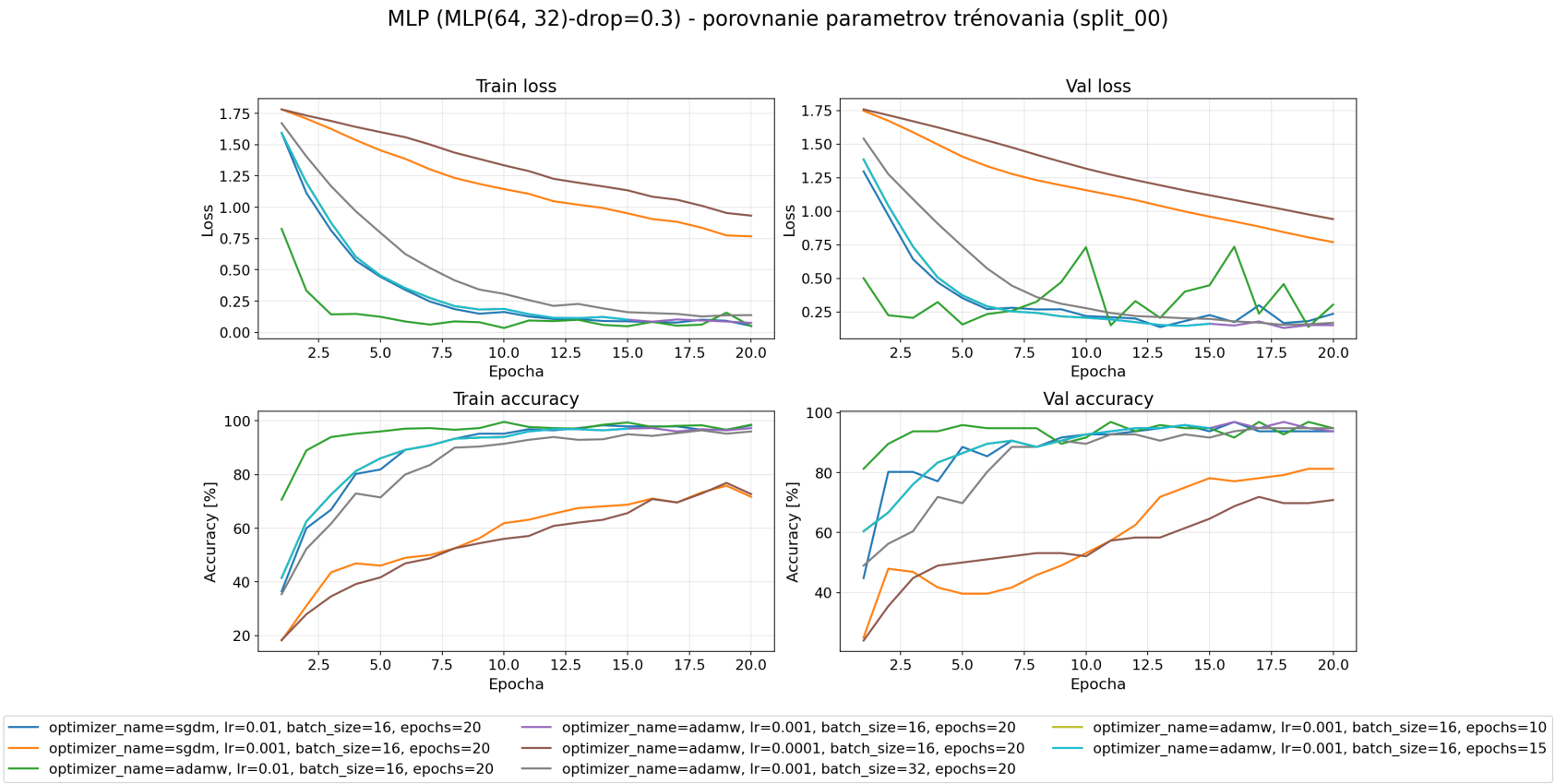
2.2 MLP

Viacvrstvový perceptrón dostáva rovnakých 54 príznakov. Testované boli štyri štruktúry (jedna skrytá vrstva 32 neurónov; 64–32; 64–32 bez Dropoutu; 128–64–32 s Dropoutom 0,4) a osem kombinácií parametrov trénovania (optimalizátor SGD s momentom vs. AdamW, learning rate 10^{-2} až 10^{-4}, veľkosť dávky 16 a 32, počet epoch 10–20).

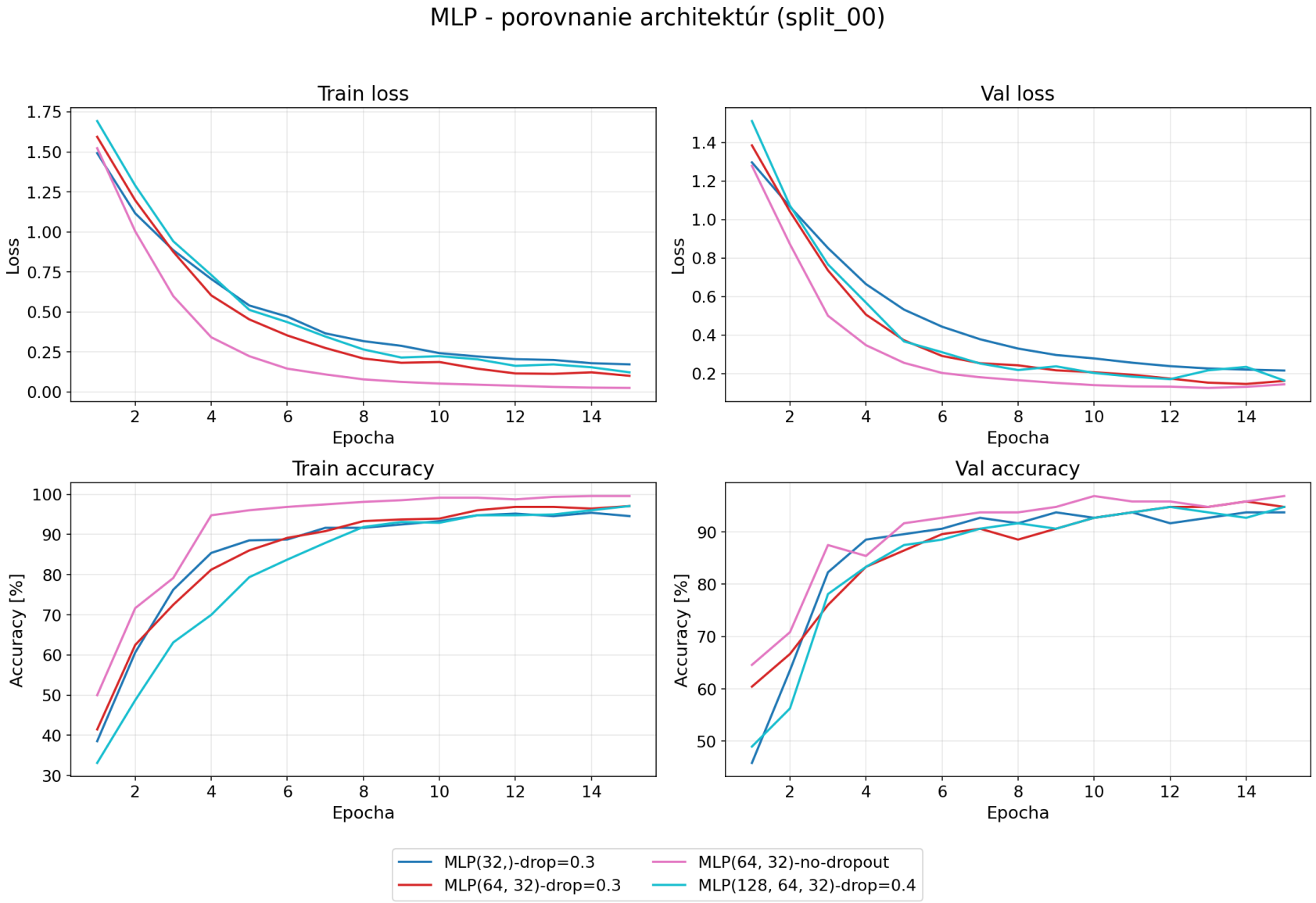
Z porovnania parametrov trénovania (Obr. 2) vyplynuli dva závery. Optimalizátor AdamW konverguje spoľahlivejšie než SGD s momentom a je podstatne menej citlivý na voľbu learning rate. Samotný learning rate je najdôležitejší parameter – pri 10^{-4}

model po 20 epochách ani zďaleka nedokonverguje (70,8 %), pri 10^{-3} až 10^{-2} dosahuje okolo 94 %.

Porovnanie štruktúr (Obr. 3) ukázalo, že najlepšie obstála sieť 64–32 *bez* Dropoutu. Pri takto malej sieti a krátkom trénovaní (15 epoch) Dropout skôr spomaľuje učenie, než by bránil preučeniu – regularizáciu tu dostatočne zabezpečuje už samotná veľkosť siete a nízky počet epoch.



Obr. 2: MLP – priebeh trénovania pri rôznych parametroch trénovania



Obr. 3: MLP – porovnanie štruktúr siete vrátane vplyvu Dropout vrstvy

3 Modely s automatickou extrakciou príznakov

3.1 CNN

Konvolučná sieť pracuje priamo so surovým oknom (400×6). Architektúru tvoria bloky `Conv1d` → `BatchNorm` → `ReLU` → `MaxPool`, po ktorých nasleduje Global Average Pooling a dve plne prepojené vrstvy.

Kľúčové zistenie z porovnania architektúr (Obr. 4) sa týka práve poolingu: varianta s `Flatten` namiesto Global Average Poolingu dosiahla len 88,5 % oproti 96,5 %. Flatten totiž vedie na vstup do plne prepojenej vrstvy s rozmerom „počet filtrov × zvyšná dĺžka“, teda rádovo tisíce parametrov navyše, čo je pri 480 trénovacích oknách priama cesta k preučeniu. Global Average Pooling tento rozmer zredukuje na počet filtrov a zároveň robí model necitlivým na to, kde presne v okne sa daný vzor vyskytol. Prínosná bola aj BatchNorm vrstva (+2 p. b.), zväčšenie siete na tri bloky už nie.

3.2 LSTM

3.2.1 Prvá verzia a jej zlyhanie

Prvá, „učebnicová“ implementácia posielala do siete okno ako sekvenciu 400 časových krokov po 6 kanáloch a klasifikovala z posledného skrytého stavu h_n. Dosiahla len **58,1 %** úspešnosti na testovacích dátach, zatiaľ čo ostatné tri modely sa pohybovali okolo 95 %. Rozbor ukázal tri príčiny:

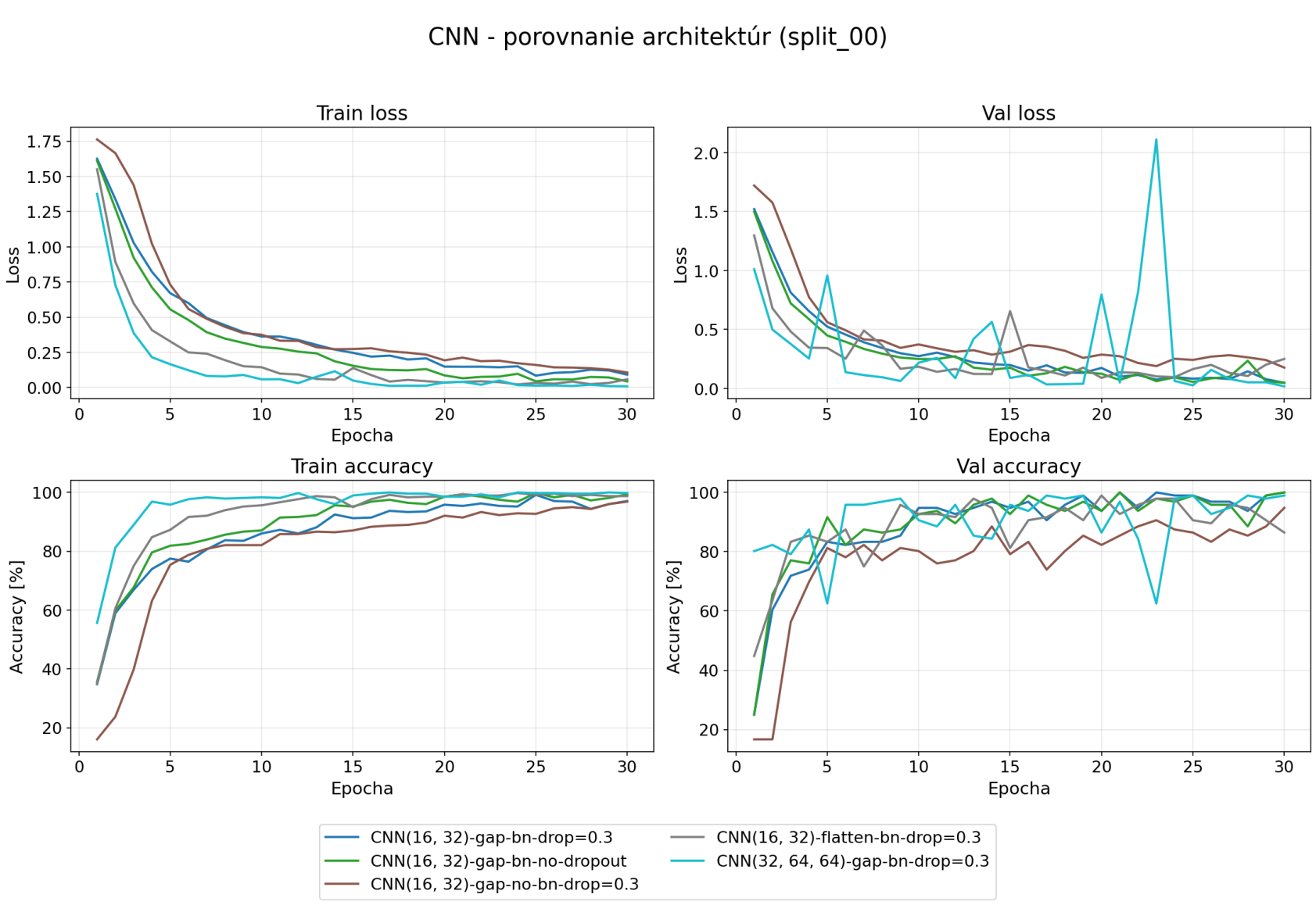
1. **Príliš dlhá sekvencia.** Cez 400 krokov sa gradient pri spätnom šírení rozptýli a sieť sa pri 480 trénovacích oknách nestihne naučiť užitočnú reprezentáciu.

2. **Nevhodný spôsob čítania výstupu.** Terén sa v signáli prejavuje ako *textúra vibrácií* rozložená po celom okne, nie ako jedna udalosť na jeho konci. Posledný skrytý stav h_n je však ovplyvnený hlavne poslednými vzorkami, takže väčšina okna sa fakticky zahodí.

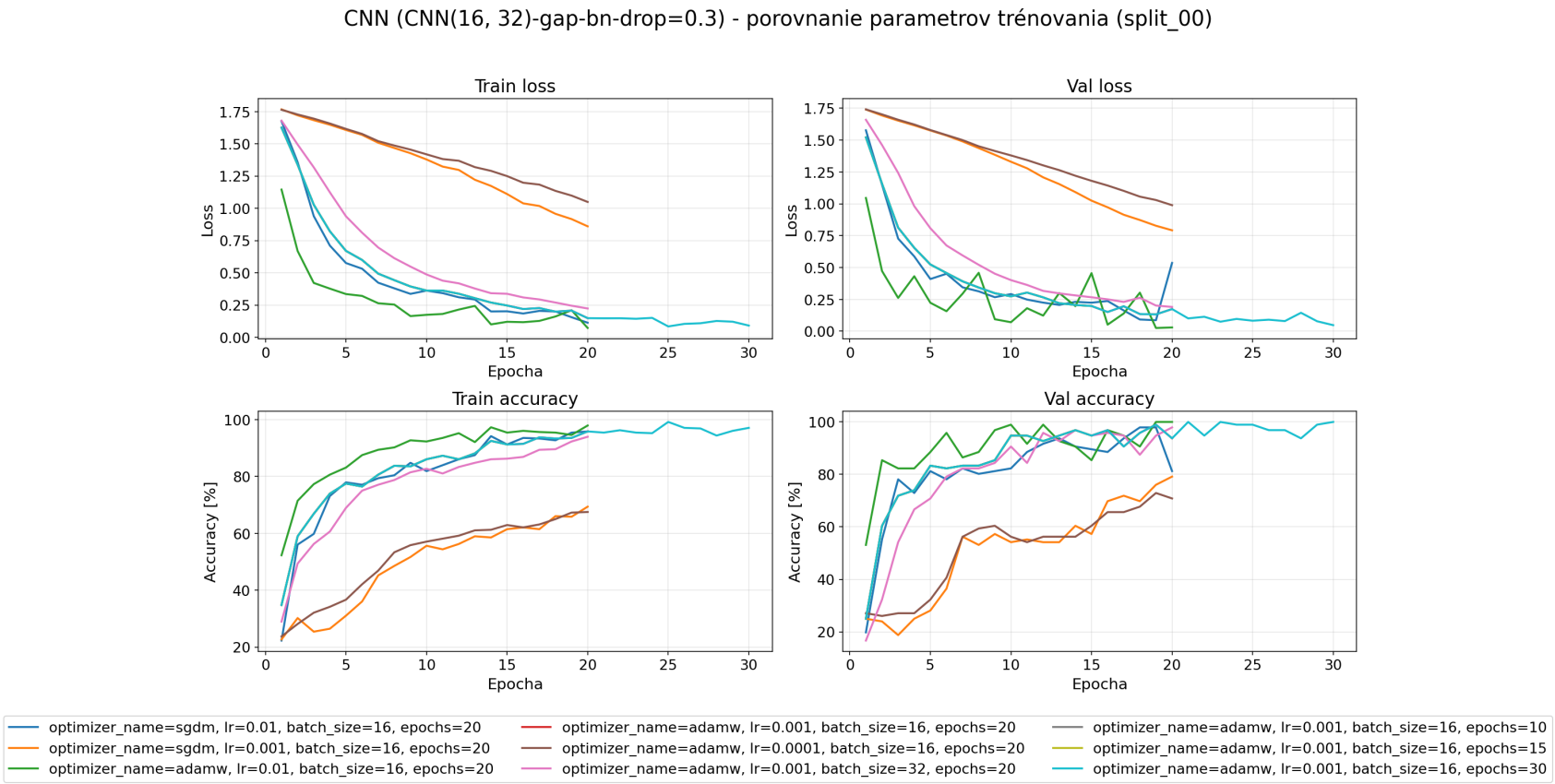
3. **Učenie sa nepodstatnej informácie.** Okno je stacionárny úsek vibrácií; to, v ktorej fáze signálu okno začína, o teréne nevypovedá nič, sieť sa však túto fázu učí spolu s užitočnou informáciou.

3.2.2 Úpravy modelu

Na odstránenie uvedených príčin boli do modelu doplnené tri mechanizmy, ktorých prínos bol samostatne overený.



Obr. 4: CNN – porovnanie architektúr



Obr. 5: CNN – priebeh trénovania pri rôznych parametroch trénovania

Pooling cez čas. Namiesto posledného skrytého stavu sa klasifikuje zo spojenia *priemeru* a *maxima* výstupov LSTM cez všetky časové kroky. Priemer zachytáva celkovú textúru okna, maximum výrazné jednorazové nárazy (kamienky, dlažba). Táto jediná zmena zvýšila úspešnosť z 57,5 % na 89,8 % (Obr. 6) a je teda hlavnou príčinou zlepšenia.

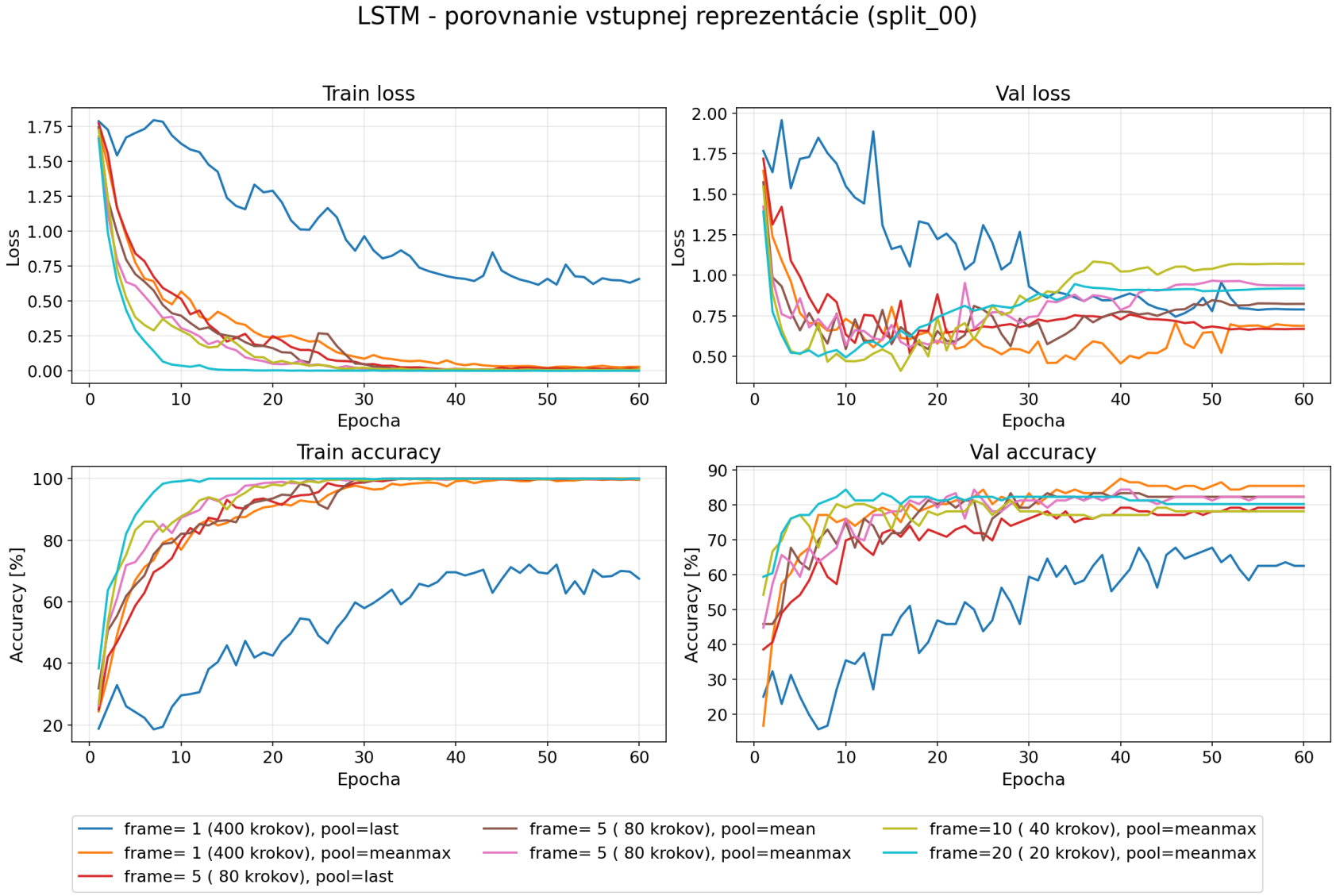
Rámcovanie vstupu. Okno sa dá preskupiť tak, že jeden časový krok siete obsahuje k po sebe idúcich vzoriek zo všetkých kanálov – napríklad $(400, 6) \to (80, 30)$ pre $k = 5$. Ide o čistý `reshape`, nie o ručne počítané príznaky, takže charakter automatickej extrakcie zostáva zachovaný. V kombinácii s poolingom cez čas však už rámcovanie ďalšie zlepšenie neprinieslo (88,5 % pre $k = 5$ oproti 89,8 % pre $k = 1$), preto finálny model pracuje s plnou sekvenciou. Výrazne pomáha len vtedy, keď sa klasifikuje z posledného skrytého stavu (57,5 % $\to$ 80,8 %). Možnosť ostala v kóde ako parameter – pri $k = 5$ je tréning približne päťkrát rýchlejší za cenu asi jedného percentuálneho bodu.

Augmentácia trénovacích dát. Na každú trénovaciu dávku sa aplikuje náhodný cyklický posun okna v čase, náhodné preškálovanie amplitúdy zvlášť pre každý kanál ($\pm$10 %) a slabý gaussovský šum. Cyklický posun priamo rieši tretiu príčinu zlyhania – posunuté okno je stále to isté meranie toho istého terénu, takže sa sieť prestane učiť konkrétnu fázu signálu. Škálovanie amplitúdy simuluje mierne odlišné uchytenie alebo citlivosť snímača. Augmentácia zvýšila úspešnosť z 89,8 % na 92,3 % (Obr. 7), pričom najväčší podiel na tom má práve časový posun.

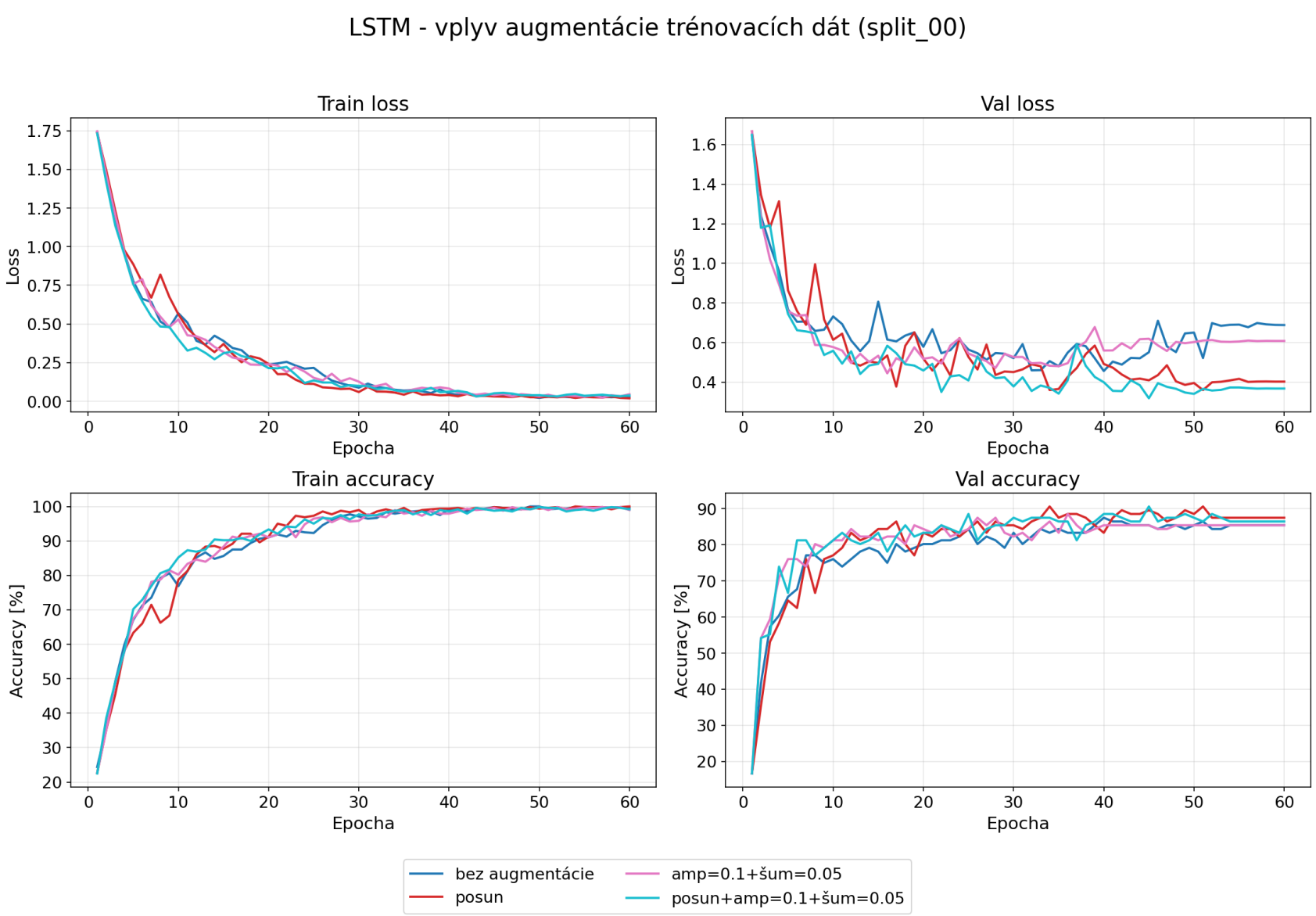
3.2.3 Ladenie parametrov a štruktúry

Nad takto upraveným modelom prebehlo porovnanie parametrov trénovania (Obr. 8). Potvrdil sa rovnaký obraz ako pri MLP a CNN – AdamW jednoznačne poráža SGD s momentom (92,3 % oproti 85,4 %) a príliš malý learning rate 10^{-4} nestačí (73,8 %). Novým zistením je, že model s augmentáciou znesie *podstatne dlhšie trénovanie*: zatiaľ čo pri 30 epochách dosahoval 87,5 %, pri 120 až 200 epochách 92,7 %. Je to očakávané – augmentácia robí každú epochu trochu inou, takže sa model nepreučí ani po 200 epochách.

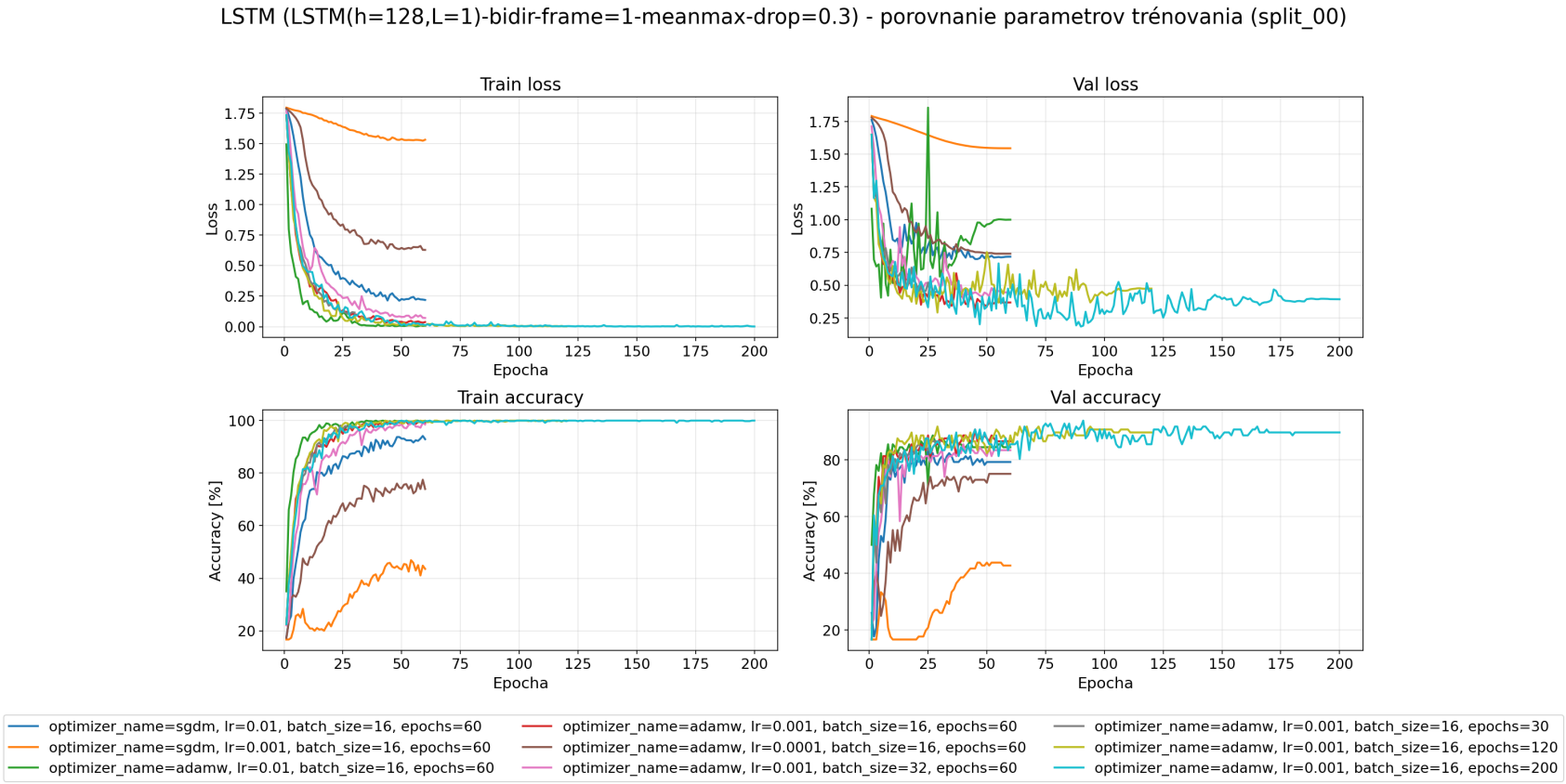
Pri porovnaní štruktúr (Obr. 9) dosiahla najlepší výsledok obojsmerná LSTM so 128 skrytými jednotkami v jednej vrstve *bez* Dropoutu (95,2 %). Vplyv Dropoutu je tu dobre viditeľný: bez neho 95,2 %, s hodnotou 0,3 dosiahol model 92,7 % a s hodnotou 0,5 len 92,1 %. Vysvetlenie je rovnaké ako pri MLP – úlohu regularizátora už preberá augmentácia dát, takže ďalšie tlmenie aktivácií model iba oslabuje. Druhá vrstva LSTM



Obr. 6: LSTM – porovnanie vstupnej reprezentácie a spôsobu čítania výstupu



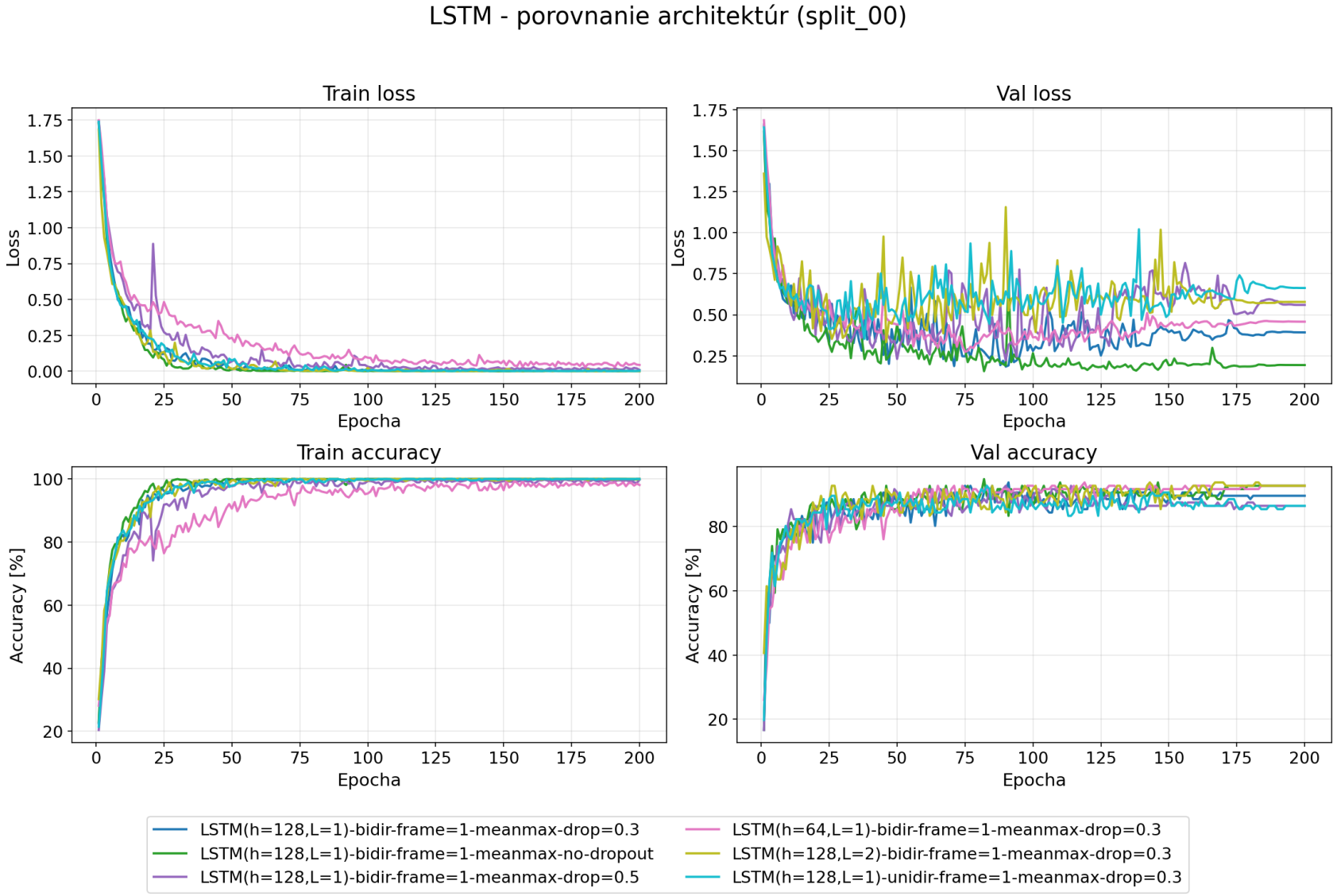
Obr. 7: LSTM – vplyv augmentácie trénovacích dát



Obr. 8: LSTM – priebeh trénovania pri rôznych parametroch trénovania

ani zmenšenie siete na 64 jednotiek neprispeli, obojsmerné spracovanie pridalo približne 1,7 p. b.

Finálna konfigurácia je teda: obojsmerná LSTM (128 jednotiek, 1 vrstva), pooling priemer + maximum cez čas, bez Dropoutu, optimalizátor AdamW s learning rate 10^{-3}, dávka 16, 200 epoch s kosínusovým rozvrhom learning rate a augmentácia (posun + amplitúda + šum).



Obr. 9: LSTM – porovnanie štruktúr siete vrátane vplyvu Dropout vrstvy

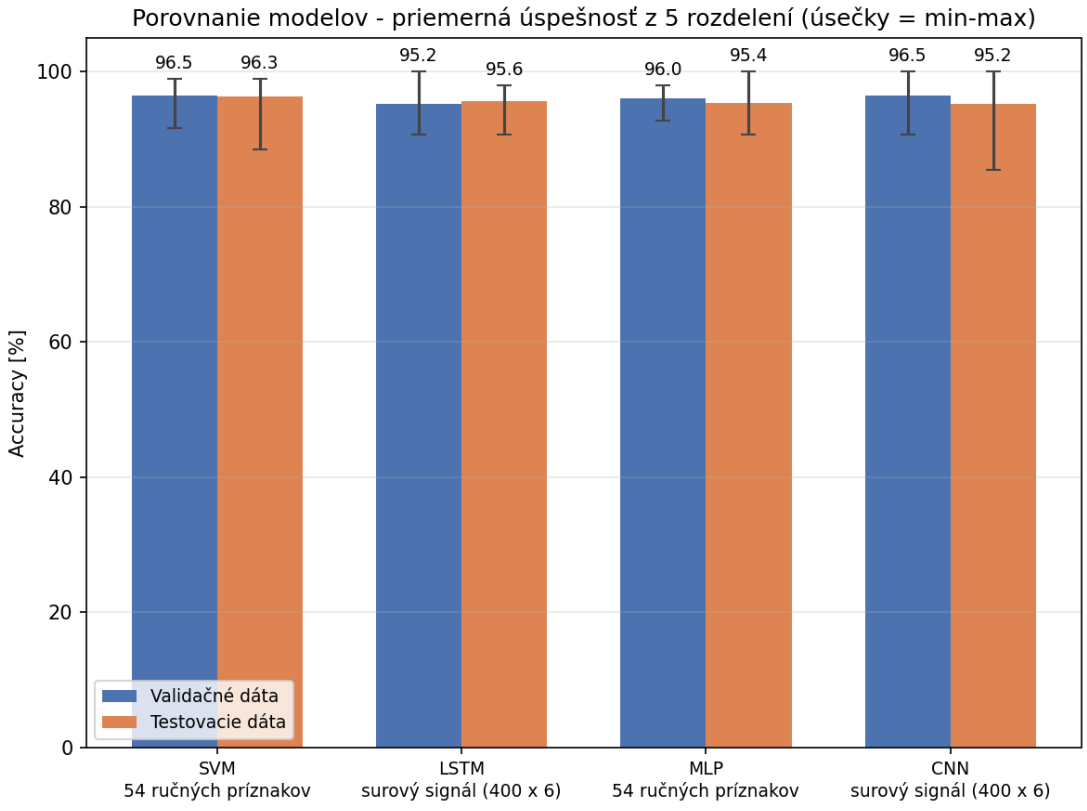
4 Výsledky a porovnanie

4.1 Súhrnné porovnanie modelov

Každý model bol v najlepšej konfigurácii spustený na 5 nezávislých rozdeleniach dát. Tabuľka 2 uvádza priemer a smerodajnú odchýlku metrík z týchto piatich behov, Obr. 10 to isté vizuálne aj s rozpätím minimum–maximum.

Tabuľka 2: Porovnanie modelov – priemer ± smerodajná odchýlka z 5 rozdelení

Model	Vstup	VAL acc	TEST acc	TEST prec	TEST recall
SVM	54 príznakov	96,46 ± 2,52	**96,25 ± 3,88**	96,99	96,25
LSTM	surový signál	95,21 ± 3,40	95,63 ± 2,59	95,91	95,63
MLP	54 príznakov	96,04 ± 2,02	95,42 ± 3,27	96,34	95,42
CNN	surový signál	96,46 ± 3,46	95,21 ± 5,34	96,30	95,21



Obr. 10: Porovnanie modelov – priemerná úspešnosť z 5 rozdelení, úsečky vyznačujú rozpätie minimum–maximum

Tabuľka 3: Rozptyl testovacej úspešnosti naprieč piatimi rozdeleniami

Model	min	max	priemer	std	Najlepšia konfigurácia
SVM	88,54	98,96	96,25	3,88	`linear, C=1.0`
LSTM	90,62	97,92	95,63	**2,59**	`h=128, bidir, meanmax`
MLP	90,62	100,00	95,42	3,27	`64-32, no dropout`
CNN	85,42	100,00	95,21	5,34	`16-32, GAP, BN`

4.2 Diskusia

Ručné príznaky vs. automatická extrakcia. Rozdiely medzi modelmi (95,2 % až 96,3 %) sú menšie než smerodajné odchýlky jednotlivých modelov (2,6 až 5,3 p. b.). Na týchto dátach teda **žiadny z prístupov nie je preukázateľne lepší** – tvrdenie, že „SVM vyhralo“, by nebolo štatisticky podložené. Je to pochopiteľné: 54 ručných príznakov (energia a hrubá frekvenčná charakteristika vibrácií) zachytáva prakticky všetko, čo je v jednosekundovom okne pre rozlíšenie povrchu podstatné, takže automatická extrakcia nemá kde získať náskok.

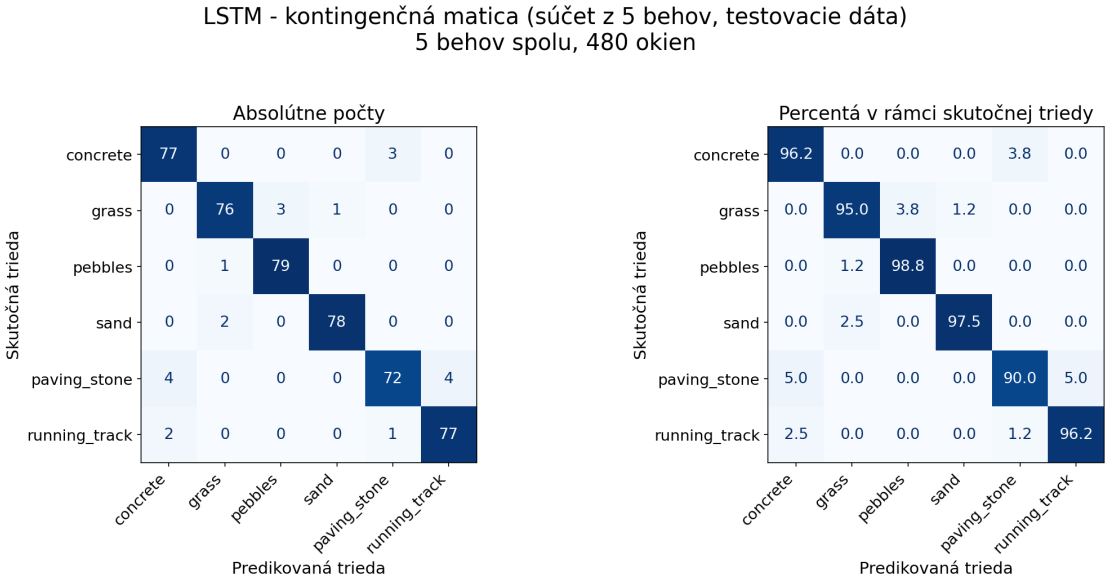
Stabilita naprieč rozdeleniami. Zaujímavejší než priemer je rozptyl. Najstabilnejší je LSTM (std 2,59), najmenej stabilná je CNN (std 5,34, najhorší beh 85,4 %). Pri iba 12 testovacích meraniach na jeden beh stačí niekoľko nesprávne klasifikovaných okien na výkyv o niekoľko percentuálnych bodov, takže nízky rozptyl je z praktického hľadiska cennejší než o pol percenta vyšší priemer. Nízky rozptyl LSTM pripisujeme augmentácii, ktorá model núti spoliehať sa na vlastnosti nezávislé od konkrétneho merania.

Cena za výsledok. Modely sa výrazne líšia výpočtovou náročnosťou. SVM sa natrénuje v sekundách na CPU, MLP za desiatky sekúnd, CNN za jednotky minút a finálny LSTM (200 epoch nad sekvenciou 400 krokov, GPU) trvá pri prechode všetkých porovnaní približne 45 minút. Pri prakticky zhodnej úspešnosti je teda SVM nad ručnými príznakmi jednoznačne najhospodárnejšou voľbou a LSTM najdrahšou.

4.3 Kontingenčné matice

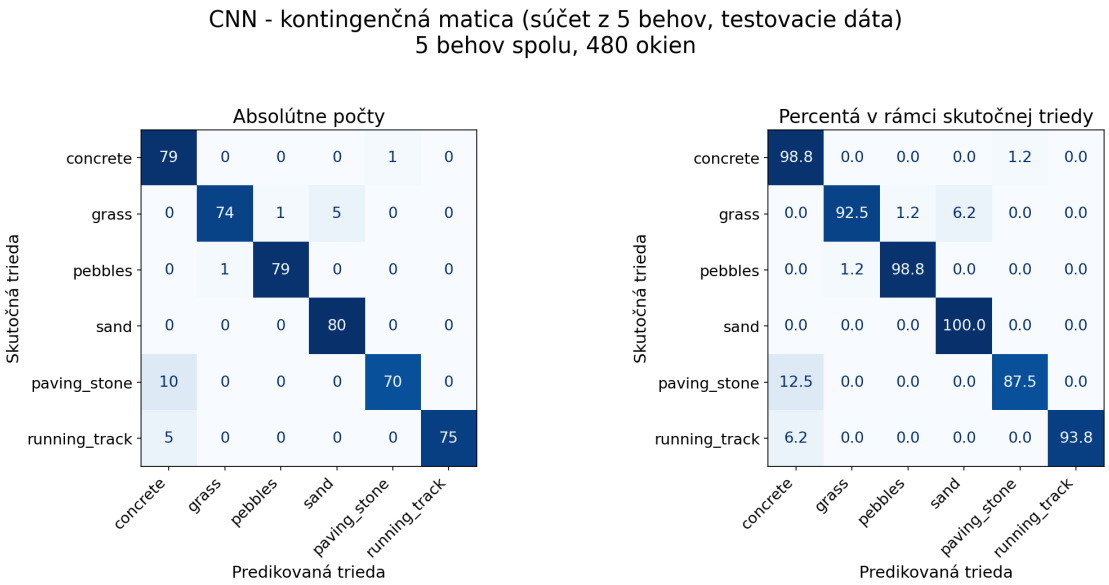
Pre každý model sa teraz z finálnych 5 behov ukladá kompletná sada kontingenčných matíc do `results/<model>/`: jedna pre **každý jednotlivý beh** (`runs/split_XX_confusion.png` – spolu 5), jedna **súčtová** (spojenie predikcií zo všetkých 5 behov, teda $5 \times 96 = 480$ testovacích okien) a jedna pre **najlepší beh** (podľa testovacej accuracy). Predtým sa ukladala len jedna matica, a to z ľubovoľne posledného behu (`split_04`) – čo pri variabilite medzi behmi (kapitola 4) skresľovalo obraz o chybách modelu.

Nižšie sú preto súčtové matice (najreprezentatívnejšie – pokrývajú všetky testovacie meranie naprieč piatimi rozdeleniami), každá v absolútnych počtoch aj v percentách v rámci skutočnej triedy.

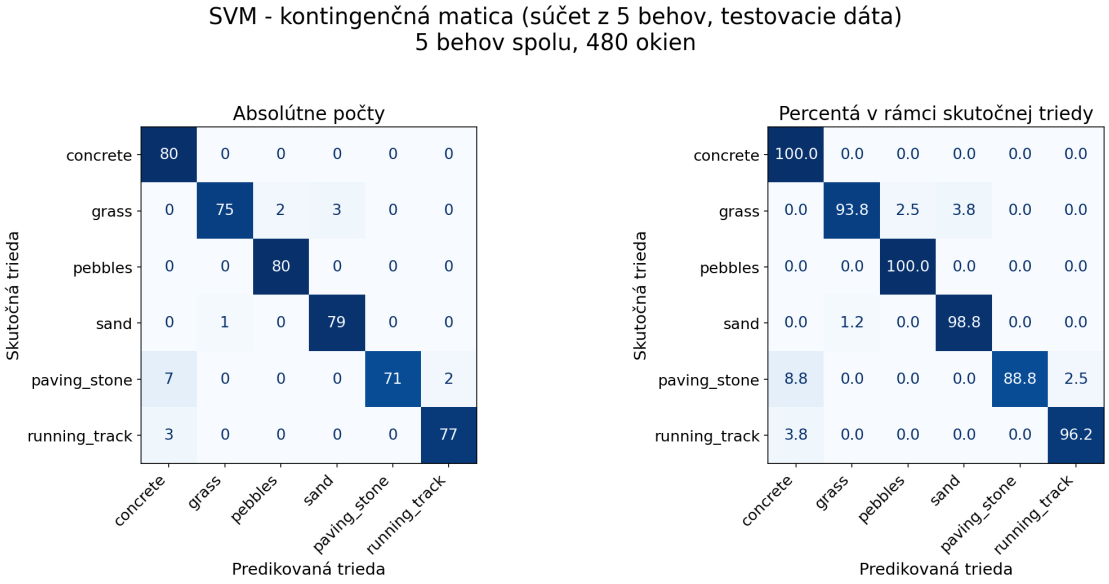


Obr. 11: LSTM – súčtová kontingenčná matica (5 behov spolu, 480 testovacích okien)

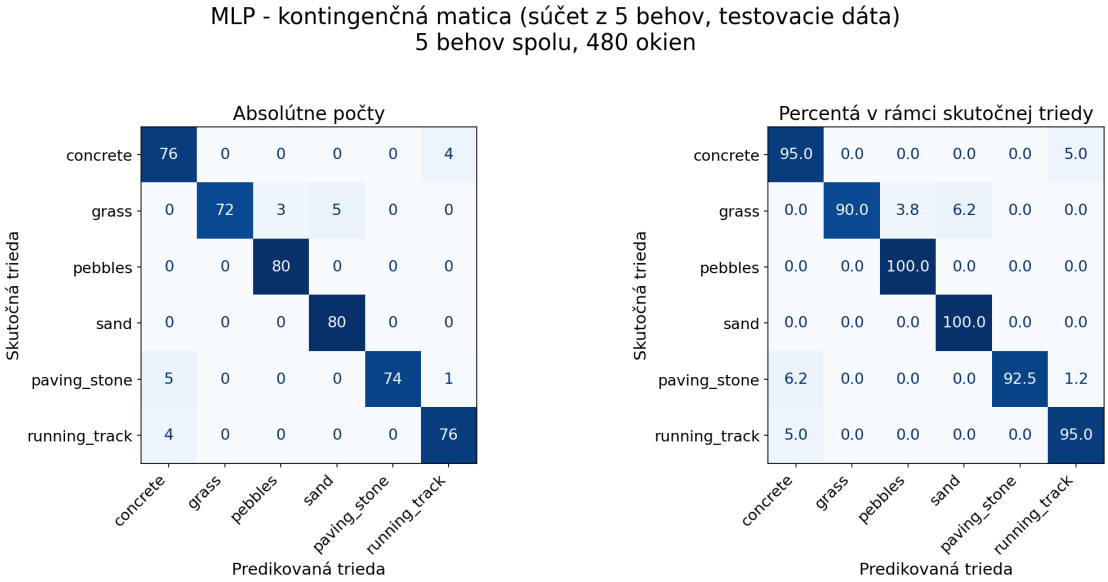
Pri LSTM sú tri triedy (tráva, kamienky, piesok) rozpoznané prakticky bezchybne. Chyby sa sústreďujú medzi *paving_stone*, *concrete* a *running_track* – teda medzi tri tvrdé, rovné a hladké povrchy, ktoré budia podobne nízke a pravidelné vibrácie. Naopak sypké a nerovné povrchy majú výrazne odlišný charakter signálu a nemýlia sa. Rovnaký vzor chýb sa objavuje aj u ostatných modelov, čo naznačuje, že ide o vlastnosť dát, nie konkrétneho klasifikátora.



Obr. 12: CNN – súčtová kontingenčná matica (5 behov spolu, 480 testovacích okien)



Obr. 13: SVM – súčtová kontingenčná matica (5 behov spolu, 480 testovacích okien)

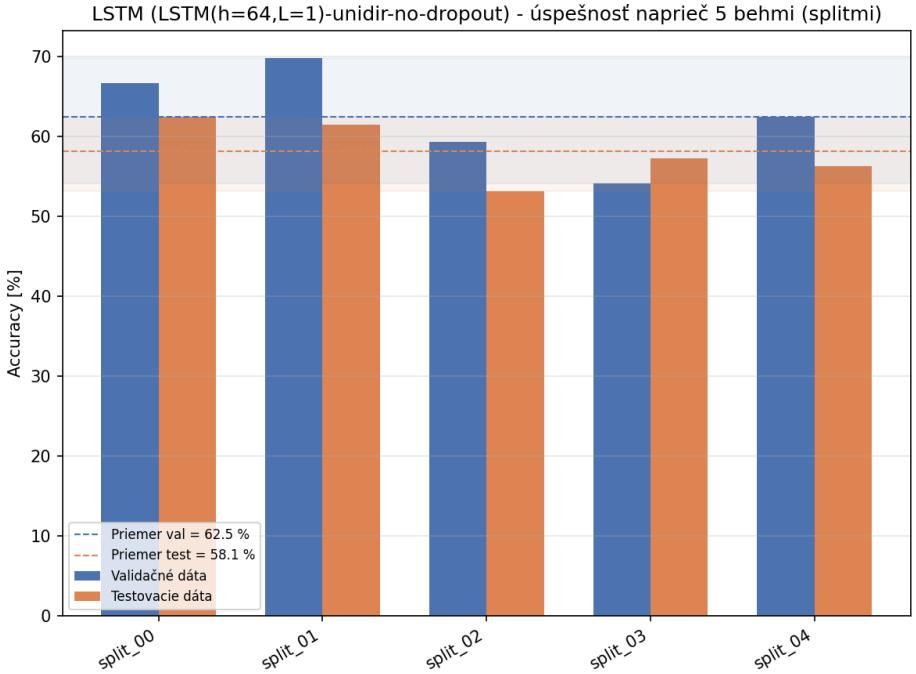


Obr. 14: MLP – súčtová kontingenčná matica (5 behov spolu, 480 testovacích okien)

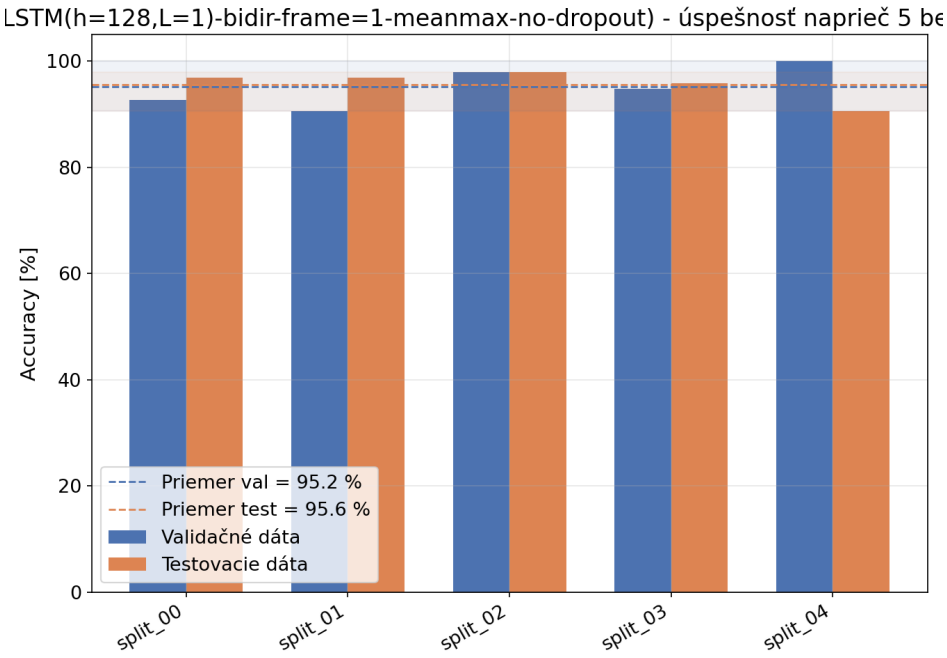
Matica **najlepšieho** behu (spolu s maticami všetkých 5 jednotlivých behov) je uložená pri každom modeli ako `<model>_confusion_matrix.png` (resp. `runs/split_XX_confusion.png`) – v texte sa neuvádza zvlášť, keďže ukazuje len jeden (priaznivo vybraný) beh, zatiaľ čo súčtová matica vyššie je štatisticky robustnejšia.

4.4 Prínos prepracovania LSTM

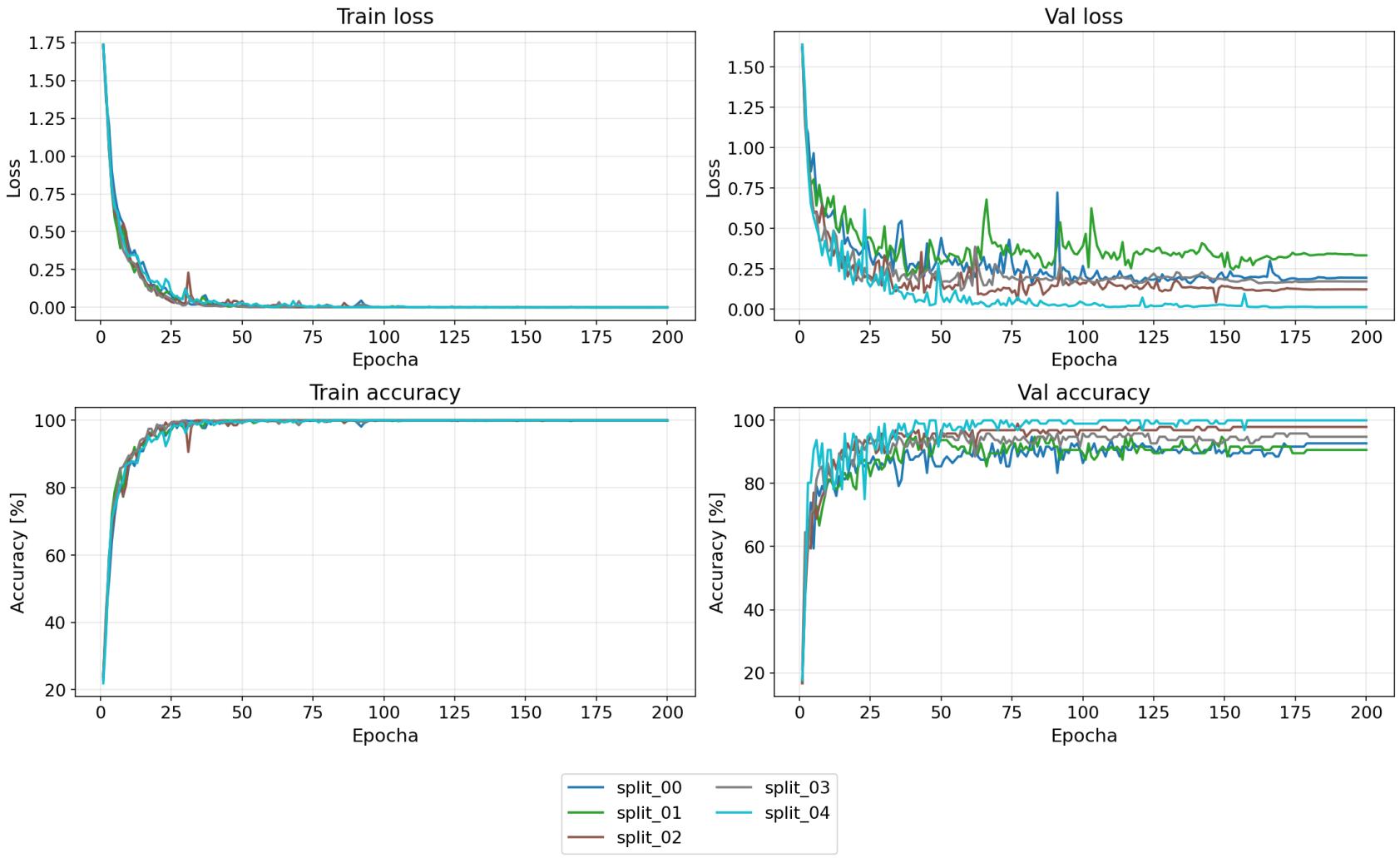
Obr. 15 a Obr. 16 porovnávajú úspešnosť pôvodnej a upravenej verzie LSTM naprieč piatimi rozdeleniami. Priemerná testovacia úspešnosť vzrástla z 58,13 % na 95,63 %, teda o 37,5 p. b., pričom sa súčasne znížil rozptyl (3,45 → 2,59) – model je nielen presnejší, ale aj stabilnejší.



Obr. 15: Pôvodná verzia LSTM – úspešnosť naprieč 5 rozdeleniami (priemer 58,13 %)



Obr. 16: Upravená verzia LSTM – úspešnosť naprieč 5 rozdeleniami (priemer 95,63 %)



Obr. 17: Upravená verzia LSTM – priebeh trénovania naprieč 5 rozdeleniami

5 Poznámky k metodike a obmedzenia

Pre korektnosť interpretácie výsledkov je potrebné uviesť niekoľko obmedzení použitého postupu.

- **Rozsah dát.** 84 meraní a 480 trénovacích okien je veľmi málo. Pri 96 testovacích oknách zodpovedá jedno nesprávne klasifikované okno približne 1 p. b. úspešnosti, preto treba rozdiely menšie než niekoľko percentuálnych bodov považovať za šum.
- **Výber konfigurácie.** Pri CNN a LSTM sa najlepšia konfigurácia vyberala podľa priemeru validačnej úspešnosti zo všetkých 5 rozdelení, pri SVM a MLP len podľa `split_00`. Porovnanie modelov medzi sebou je preto mierne v neprospech SVM a MLP – ich hyperparametre boli ladené na hlučnejšom odhade.
- **Kontingenčné matice** v kapitole 4.3 sú súčtové (zo všetkých 5 behov, 480 okien), takže na rozdiel od matice jedného behu nezávisia od toho, ktoré meranie náhodou padlo do ktorého rozdelenia.
- **Testovacie dáta** neboli použité pri žiadnom rozhodovaní o modeli – všetky výbery prebehli výlučne na validačnej množine. Uvádzané testovacie metriky sú teda nezaujatým odhadom.

6 Implementácia

Projekt je napísaný v Pythone (NumPy, pandas, scikit-learn, PyTorch, matplotlib). Štruktúra zdrojových súborov zodpovedá jednotlivým krokom spracovania:

Tabuľka 4: Štruktúra zdrojových súborov

Súbor	Obsah
`config.py`	centrálna konfigurácia (cesty, frekvencie, okno, semienko)
`data_loading.py`	načítanie CSV a katalóg meraní
`splitting.py`	rozdelenie na train/val/test na úrovni meraní
`windowing.py`	segmentácia na okná 1 s s posunom 0,5 s
`features.py`	výpočet 54 časových príznakov
`models/svm_model.py`	SVM
`models/mlp_model.py`	MLP + spoločná tréningová slučka
`models/cnn_model.py`	1D CNN + normalizácia po kanáloch
`models/lstm_model.py`	LSTM + augmentácia
`models/evaluate.py`	metriky, grafy, ukladanie logov
`models/compare_models.py`	súhrnné porovnanie všetkých modelov

Všetky modely používajú rovnaké funkcie z `evaluate.py` na výpočet metrík, takže ich výsledky sú priamo porovnateľné. Každý beh si navyše ukladá kompletný výpis z konzoly do súboru `results/<model>/<model>_training_log.md`, z ktorého skript `compare_models.py` bez opätovného trénovania poskladá súhrnnú tabuľku, CSV a graf použitý v tejto dokumentácii (Obr. 10).

Pre finálnu konfiguráciu každého modelu (5 behov) sa okrem toho do `results/<model>/runs/` ukladá kompletná surová stopa každého behu: predikcie na validačných aj testovacích dátach (`split_XX_predictions.npz`), priebeh trénovania po epochách (`split_XX_history.json`, pri sieťových modeloch), čas trénovania a metriky (`split_XX_meta.json`) a súhrnná tabuľka `runs.csv` s riadkami pre všetkých 5 behov aj s min/max/priemer/std. Vďaka tomu sa dá čokoľvek dopočítať dodatočne (napr. súčtová kontingenčná matica v kapitole 4.3) bez opätovného trénovania.

Záver

Cieľom zadania bolo klasifikovať šesť typov terénu zo signálov akcelerometra a gyroskopu a porovnať prístup s ručne navrhnutými príznakmi (SVM, MLP) s prístupom, kde si príznaky extrahuje sieť sama (LSTM, CNN).

Všetky štyri modely dosiahli na testovacích dátach priemernú úspešnosť medzi 95,2 % a 96,3 %, pričom rozdiely medzi nimi sú menšie než smerodajné odchýlky jednotlivých modelov. Na tomto datasete teda ani jeden z porovnávaných prístupov nie je preukázateľne lepší. Vzhľadom na to, že SVM nad 54 ručnými príznakmi dosiahlo rovnaký výsledok za zlomok výpočtového času, je pre túto úlohu najhospodárnejšou voľbou. Najstabilnejším modelom naprieč rozdeleniami bol upravený LSTM (smerodajná odchýlka 2,59 p. b.).

Najpoučnejšou časťou práce bolo prepracovanie modelu LSTM. Jeho prvá verzia, ktorá posielala do siete 400 časových krokov a klasifikovala z posledného skrytého stavu, dosahovala len 58,1 %. Analýza ukázala, že problém nebol v kapacite siete ani v parametroch trénovania, ale v *spôsobe, akým model čítal signál*. Nahradenie posledného skrytého stavu poolingom cez celú sekvenciu zvýšilo úspešnosť o vyše 32 p. b., augmentácia trénovacích dát (najmä cyklický časový posun) pridala ďalšie percentuálne body a odstránenie Dropoutu, ktorého úlohu už prevzala augmentácia, dovŕšilo výsledok na 95,6 %.

Z porovnania architektúr vyplynul spoločný motív pre všetky neurónové modely: pri 480 trénovacích oknách rozhodovala regularizácia a vhodná redukcia rozmeru, nie veľkosť siete. Potvrdzuje to Global Average Pooling pri CNN (+8 p. b. oproti variante s Flatten), pooling cez čas pri LSTM aj to, že najlepšie štruktúry MLP aj LSTM boli tie bez Dropout vrstvy – v oboch prípadoch preto, že regularizáciu už zabezpečoval iný mechanizmus.

Ako ďalší krok by dávalo zmysel overiť modely na väčšom množstve meraní, prípadne vyhodnotiť predikcie agregované cez celé meranie (hlasovanie z ôsmich okien) namiesto jednotlivých okien – v praktickom nasadení robot rozhoduje o type povrchu priebežne z viacerých po sebe idúcich okien, nie z jediného.